

卖家第二大脑

GBrain 实战笔记

让 Agent 像老员工一样懂你的生意

V2.0

关于作者

Zach，公众号「Zach的进化笔记」主理人。

计算机专业没学好所以一毕业就干了亚马逊的运营。

还不是老板，所以只能开玩笑说自己是「亿级操盘手」，也确实是在公司用两年从 0 做到年销售额接近 2 亿 GMV。

走的是白帽精品路线。擅长的是站内广告、DSP、季节性 FBA 库存管理，是个运营人，而不是程序员。

正在做一件自己觉得挺兴奋的事：搭一家 AI Native 的跨境电商公司。过程中沉淀的 Skill、MCP、知识库实践，能开源的都开源。

这本书不是教程。是我在真实搭建跨境电商 AI 第二大脑时形成的判断——有些对，有些可能会改，但都是真的在用。



公众号：Zach的进化笔记

回复“第二大脑”加读者群

核心输出：开源通用 Skill / MCP，输出电商公司落地 AI 的想法和判断



知识星球：Zach的跨境AI资产库

并非交付课程

深入交流 AI 落地到企业的卡点，在星球内沉淀 Skill、知识库等未来的 AI 资产

00

前言：为什么跨境电商团队迟早要建设 AI 第二大脑，以及怎么做

思考

这本书到底要解决什么？

跨境电商 AI 化的下一个瓶颈不是工具不够多，而是 Agent 没有业务记忆。

五条底层判断

在展开之前，先亮底牌。这本书的所有内容建立在这五条判断之上——如果你同意其中三条以上，值得往下读；如果每一条都存疑，更值得——后面的章节就是在用跨境电商的真实场景回答“凭什么”。

- 1 **Skill 和 MCP 有明显的能力天花板。** 没有业务记忆层，AI 长期做不到真正好用。工具再多、连接再强，Agent 每次还是从零开始判断。
- 2 **LLM Wiki 是方法论，GBrain 是这个方法论目前的最佳答案。** 卡帕西提出了把知识整理成 LLM 能读的页面网络的框架，GBrain 把它变成了可安装、可运行、可维护的系统。
- 3 **知识库不是资料库。** 把文件塞进 AI 不等于给它记忆。记忆是结构化的判断、关系和时间线，不是一堆搜索结果。
- 4 **这件事难在理解，不难在技术。** 安装 GBrain 半小时就能跑。但理解“什么该写成记忆页、什么只是噪声”——这需要业务判断，不是技术能力。
- 5 **人和 Agent 的长期协作，需要 Agent 拥有你的个人第二大脑。** 业务记忆服务组织，个人记忆服务你自己。两层记忆，才是完整的 AI Native 第二大脑。

这不是 GBrain README 的中文转述。这是我在真实搭建跨境电商 AI 第二大脑过程中形成的判断，有些可能会随着 GBrain 的迭代而变化，但到目前为止我认为它们成立。

Skill 和 MCP 的天花板在哪里

你可能已经感觉到了——AI 工具越来越多，Skill 越来越强，但实际业务中 Agent 给你的建议还是经常“差口气”。

它写的广告分析看上去像模像样，却不知道你上个月已经因为库存不足暂停了这条产品线。它能帮你调 CPC，却不知道你们内部“利润率低于 15% 的链接不再加预算”这条规则。它能把复盘写完，明天还是像新人一样重新开始——不记得上次“加大投放”被执行后结果如何，也不知道团队为什么形成了“库存覆盖不足 21 天时暂停放量”的规则。

这不是 AI 不聪明。Skill 解决的是“怎么做”——怎么跑广告分析、怎么写 listing、怎么复盘。MCP 解决的是“能连什么”——能读报表、能调 API、能查数据。但“为什么这样做”——上次的判断、上次的结果、团队积累的经验——这些全丢了。

天花板不是工具不行，而是没有记忆层。Agent 缺的不是能力，是三层东西：

缺上下文。 Agent 每次从零开始，不知道上次的业务判断。明明告诉过它北美站转化率在下降、应该控制预算，下次问到同一个产品时它又建议加大投放。不是判断力不够，是根本不记得上次的结论。

缺关系。 Agent 能查单个产品的数据，但不知道这个产品和哪些渠道、竞品、供应链约束有关联。跨境业务的判断几乎都是跨对象的——暂停一个广告活动，可能是因为库存、竞品、利润率三方面同时出了变化。看不到这些关系，就只能给出片面建议。

缺演进。 Agent 不知道“上次为什么这样决策、后来结果如何”。没有时间线，就没有复盘能力。而跨境电商最有价值的经验恰恰是“上次踩了坑，这次怎么避开”。

所以你需要的是不是更多工具，而是一套业务记忆系统——让 Agent 带着你团队的长期判断去做事。

LLM Wiki 是方法论，GBrain 是落地答案

如果你关注 AI，大概率听过卡帕西的 LLM Wiki——把知识整理成 LLM 能读懂、能引用、能更新的 Markdown 页面网络。这个洞察很对，也是目前知道的人最多的 AI 知识库方法论。

但 LLM Wiki 是方法论，不是工具。真正落地时，搜索怎么做、关系怎么连、Agent 怎么调用、怎么防止知识腐烂——全要自己搭。对没有 AI 工程团队的跨境电商公司来说，LLM Wiki 很容易跑出 demo，却不容易变成长期可维护的系统。

GBrain 做的就是这件事。它来自 Y Combinator President & CEO Garry Tan 自己的 OpenClaw / Hermes 工作流（来源：[YC 人物页 \(https://www.ycombinator.com/people/garry-tan\)](https://www.ycombinator.com/people/garry-tan)、[X 推文 \(https://x.com/garrytan/status/2042497872114090069\)](https://x.com/garrytan/status/2042497872114090069)），把 LLM Wiki 方法论变成了一套可安装、可维护的系统——自带 34 个 skills、recurring jobs、MCP 接入，已经把 PDF、图片、语音、社媒和长文研究纳入 Agent Brain 框架。

我们选择围绕 GBrain 写这本书，不是因为它功能最多，而是因为它是目前 LLM Wiki 方法论下工程化程度最高、维护者持续投入最深的开源答案。

记忆不是搜索结果

把文件塞进 AI 不等于给它记忆。很多团队做的”知识库”，本质上是文件搜索——把一堆报表、会议纪要、竞品截图丢进去，AI 帮你搜出”包含’预算’的段落”。

但记忆是另一回事。一个能回答”上次为什么暂停北美站厨房收纳盒的广告预算”的系统，和一个只能找出包含”预算”关键词的段落的系统，差距不在检索能力，在于有没有人把判断整理成 Agent 能理解的结构化页面——带着对象、关系、时间线和当时的决策背景。

这本书花最多篇幅讲的就是：怎么把跨境电商的业务判断变成这种结构化记忆。

难在理解，不在技术

安装 GBrain 半小时就能跑起来。但理解”什么该写成记忆页、什么该留在原始文件”“哪些判断值得沉淀、哪些只是噪声”——这需要业务判断，不是技术问题。这本书花最多篇幅讲的不是怎么装，而是怎么想。

个人第二大脑

业务大脑让 Agent 带着组织的记忆做事，但这只是一半。另一半是：Agent 还需要知道你是谁——你的判断习惯、工作偏好、过往决策和长期积累的认知。

今天你每换一个模型、每开一个新对话，都在从零开始”教”AI 认识你。上周在 Claude 里梳理的竞品判断，这周到 ChatGPT 问同一个问题时它完全不知道。这不是效率问题，而是协作模式的根本缺陷。真正的人机长期协作，建立在 Agent 对你的持续理解之上——而这份理解不应该被锁在任何一个模型或平台里。

这本书的路线

路线很简单：先不搬空所有历史资料，也不追求自动化闭环。先用低风险样本建一个人能读的 Markdown brain repo，再让 GBrain sync、embed、query，最后接入 Agent 工作流做轻量 eval。先证明这套 brain 能召回对象、关系、判断和时间线，再逐步扩展。

行动

先别急着安装 GBrain。先写下你团队最近 3 个重复出现的判断问题——Agent 每次都答不出来的那种。再写下第一版 brain 应该从哪 10 条业务记录开始。

01

导读：这本书怎么读

思考

这么长一本书，不从头读可以吗？

可以。这本书要大而全，但读者不需要从头到尾读。不同角色按问题跳读。

如果你是非技术卖家老板

优先看：

- 00 前言
- 02 三条路线对比
- 03 为什么 GBrain 值得单独写一本书
- 12 Skill + MCP 的能力边界
- 20 企业级使用 GBrain
- 21 双脑架构：业务大脑与个人大脑
- 23 什么不该进入 GBrain
- 25 给不同团队的落地建议

你要解决的是判断问题：这件事值不值得做，应该投入到什么程度，哪些坑不能踩。

如果你是懂一点 AI 工具的运营负责人

优先看：

- 05 Page-first 与双读者设计
- 06 Obsidian 的位置
- 08 跨境业务知识对象

- 13 GBrain 的 Skill 能力与团队工具分工
- 14 从 20 条低风险知识样本开始
- 16 跨境业务复盘 MVP
- 18 让知识进入 Agent 工作流
- 20 企业级使用 GBrain
- 21 双脑架构：业务大脑与个人大脑

你要解决的是组织问题：飞书、Obsidian、GBrain、Codex、Skill、MCP 怎么分工。

如果你是准备搭内部 AI 工作流的执行者

优先看：

- 04 基础概念
- 05 Page-first 与双读者设计
- 07 GBrain 的位置与理解门槛
- 09 正式页、来源摘要页、治理页
- 11 Schema 和 Resolver
- 15 最小 Markdown brain repo
- 18 让知识进入 Agent 工作流
- 19 轻量 eval
- 20 企业级使用 GBrain
- 22 个人大脑实操指南

你要解决的是落地问题：目录怎么建、页面怎么写、关系怎么连、Agent 怎么查、怎么验证有用。

本书的虚构案例

全书用一个虚构的跨境电商场景串联：一个做家居收纳的亚马逊卖家团队，3 条产品线（厨房收纳、衣柜收纳、浴室配件），主要在北美站和欧洲站经营，面临供应链周期性波动和竞品低价冲量的挑战。这不是任何真实公司，但场景都来自跨境电商的常见业务判断。

章节地图

部分	章节	解决的问题
第一部分：为什么需要这本书	00-01	跨境电商为什么迟早要建设 AI 第二大脑，读者怎么跳读
第二部分：三条技术路线	02-03	RAG、LLM Wiki、GBrain 的差异，以及为什么 GBrain 值得认真理解
第三部分：理解 GBrain	04-07	基础概念、page-first、Obsidian、GBrain 的位置与理解门槛
第四部分：跨境业务知识怎么建模	08-13	业务对象、三层页面、schema、Skill / MCP、GBrain Skill 能力与边界
第五部分：实战路线	14-22	从 20 条样本到复盘 MVP、竞品分析、sync、query、eval、企业级应用
第六部分：风险与下一步	23-26	写入边界、风险、团队建议和长期判断

行动

按你的角色选 6 章先读，不要把这本书当成必须顺序阅读的教材。

第二部分：三条技术路线

02

三条路线对比：RAG、LLM Wiki、GBrain

思考

前言说了”LLM Wiki 是方法论，GBrain 是落地答案”——但如果你想更深入理解这个判断背后的技术路线差异，这一章拆开讲。

跨境电商团队要做 AI 知识库，最容易混淆三条路线。它们不是简单的新旧关系，而是把”资料、页面、长期记忆”放在不同位置。

路线	基本单位	AGENT 做什么	最适合解决
传统 RAG	chunk	查询时临时召回片段	查资料、找原文、补证据
LLM Wiki	page	把来源整理进 Markdown wiki	把知识组织成页面网络
GBrain	agent-maintained memory	对页面做检索、关系、时间线和 MCP 调用	让 Agent 长期使用业务信息

传统 RAG：能查资料，但不等于业务记忆

RAG (Retrieval-Augmented Generation, 检索增强生成) 的常见路径是 chunk-first: 把原始文档切成片段, 做 embedding, 再按相似度召回。简单说就是 AI 先搜一遍资料再回答你——类似于你问助理一个问题, 助理先翻了一遍文件夹, 然后把找到的片段拼成回答。Karpathy 在 [LLM Wiki 原文](https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f) (https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f) 里把这种模式概括为: 上传文件, 模型在查询时检索相关片段, 然后生成回答。

这条路线成熟实用, 但它解决的是资料召回, 不是业务记忆。跨境业务里, 很多看似在“查资料”的问题, 实际在追溯判断——“为什么上次暂停了北美站厨房收纳盒的广告预算”, 这个问题很难靠一个 chunk 回答, 因为判断分散在报表、会议、竞品截图、供应链状态和运营经验里。

chunk-first 的三个上限: 第一, chunk 容易丢上下文——一个片段提到“CPC 上升”, 却没提到利润、库存和当周目标。第二, embedding 擅长找相似, 不擅长找关系——它知道“流量成本上升”和“广告变贵”相近, 却不知道某个流量动作被哪次供应链约束限制。第三, RAG 默认不维护当前结论——业务知识会变, 一个渠道今天该收缩, 三周后可能该恢复。

这不是否定 RAG。RAG 适合做资料层、证据层、原文检索层。它应该成为 AI 第二大脑的一部分, 而不是承担整个系统。

LLM Wiki：好方法论，但工程化要自己补

LLM Wiki 的突破, 是把知识从 chunk 推到 page: 先把资料整理成持续更新的 Markdown 页面, 用链接和 schema 组织起来, 再让 LLM 读取这些页面。这比 RAG 更接近业务——跨境团队真正需要的不是“找一段相似文字”, 而是“理解一个产品、一个渠道、一个竞品、一个决策的历史”。

Karpathy 原文把 LLM Wiki 拆成三层和三类操作:

层 / 操作	意思	对跨境团队的翻译
Raw sources	原始资料, LLM 读取但不修改	报表、会议纪要、竞品资料、截图
Wiki	LLM 生成和维护的 Markdown 页面	产品页、渠道页、竞品页、复盘页
Schema	告诉 LLM 怎么组织 wiki 的规则	命名、目录、字段、写入流程
Ingest	新来源进入时, 写摘要、更新页面和索引	新报告进来后更新复盘页
Query	查询 wiki, 有价值的答案写回页面	问“为什么暂停预算”, 答案变成决策页
Lint	定期检查矛盾、过期、孤页、缺链接	防止知识库越写越乱

原文还强调两个导航文件：`index.md`（内容导航，帮 LLM 找相关页面）和 `log.md`（时间日志，记录变更历史）。

这些机制很强，也解释了为什么 LLM Wiki 能跑出漂亮 demo。但它仍然是方法论。它没有替你完成数据库、检索融合、typed graph、timeline、MCP、维护循环和评估体系。没有工程团队时，这些会变成长期成本。

GBrain：持续演进的 Agent Brain 框架

Garry Tan 本人在介绍 GBrain 时做了一个明确区分：这不是“RAG in a box”，而是一套完整的知识系统（knowledge system）——8 层架构协同工作，让 Agent 不只是检索资料，而是真正理解你的业务上下文（来源：[X 主帖 \(https://x.com/garrytan/status/2055670533451366479\)](https://x.com/garrytan/status/2055670533451366479)）。

GBrain 保留 Markdown 作为知识源，同时引入数据库、Hybrid Search、typed links、timeline、MCP 和维护流程。GBrain README 明确强调这些能力，正是 LLM Wiki 方法论落地后会遇到的工程问题（来源：[GBrain README \(https://github.com/garrytan/gbrain/blob/master/README.md\)](https://github.com/garrytan/gbrain/blob/master/README.md)）。

但它不只是“把 LLM Wiki 工程化”。官方 README 当前写到：项目来自 Garry Tan 自己的 OpenClaw / Hermes 工作流，安装流程会加载 34 个 skills，并配置 recurring jobs；skillpack 还提供 `media-ingest`、`book-mirror`、`brain-pdf`、`X-to-Brain`、`Voice-to-Brain` 等已经写好的能力（来源：[GBrain README \(https://github.com/garrytan/gbrain/blob/master/README.md\)](https://github.com/garrytan/gbrain/blob/master/README.md)、[GBRAIN_SKILLPACK \(https://github.com/garrytan/gbrain/blob/master/docs/GBRAIN_SKILLPACK.md\)](https://github.com/garrytan/gbrain/blob/master/docs/GBRAIN_SKILLPACK.md)）。

这意味着 GBrain 的核心不是“多了一个数据库”，而是把 memory、skills、MCP、定时任务和跨模态摄取放在同一套 Agent 工作流里。

示例

传统 RAG 能找出某份渠道报告里的异常数据；LLM Wiki 能把这次异常写成一页复盘；GBrain 则让下次 Agent 做同类复盘时，沿着产品、渠道、关键词、竞品、供应链约束，把上次判断调出来。如果团队还要处理 PDF 行业报告、竞品图片、会议录音或社媒素材，GBrain 内置 skillpack 的价值就从“技术组件”变成“可复用工作流”。

落地决策：选哪条路线

如果上面的技术对比还太抽象，下面这张表帮你做实际决策：

维度	传统 RAG	LLM WIKI (自建)	GBRAIN
团队要求	不需要专人维护 AI	需要有人持续整理 Markdown	同 LLM Wiki, GBrain 处理
首次见效时间	小时级 (上传文件, 开始提问)	天级 (写页面、连关系)	天级 (同样的页面工作 + sy
长期维护成本	低 (文件是静态的)	高 (页面不维护会腐烂)	中 (内置维护 skills 辅助)
典型失败模式	无法回答跨对象的判断问题	工程缺口累积导致 Agent 用不上	理解缺口导致页面质量低

没有哪条路线是错误的起点。如果你现在连资料都还没有集中存放，RAG 是合理的第一步。如果你已经在用 Obsidian 整理知识但 Agent 用不上，GBrain 是下一步。

行动

判断你团队现在处在哪一层：你是在查资料（RAG），在整理页面（LLM Wiki），还是已经需要 Agent 长期调用业务记忆（GBrain）？

03

为什么 GBrain 值得单独写一本书

思考

开源项目那么多，为什么 GBrain 不是普通工具介绍，而值得写成一本实战笔记？

GBrain 值得单独写，是因为它试图回答一个跨境团队迫切需要但市面上没人系统讲过的问题：LLM Wiki 里的 Markdown 页面，如何变成 Agent 长期记忆系统。

这件事对跨境电商团队很关键。卖家需要的不是漂亮知识库，而是一套能被持续使用的业务记忆：人能读懂页面、运营能在 Obsidian 里编辑和追溯、Agent 能通过 GBrain 检索和调用、Skill 能把判断流程稳定执行、新结论能回到长期记忆而不是消失在一次对话里。

GBrain 值得观察的三层

第一，维护者和使用场景很特殊。GBrain README 把它描述成支撑 Garry Tan 个人 OpenClaw / Hermes 部署的生产 brain，不是临时 demo。他在 X 上也公开将 GBrain 放在 Agent 长期记忆语境中，强调面向 10,000+ Markdown 文件的长期召回和 MIT 开源（来源：X 推文 (<https://x.com/garrytan/status/2042497872114090069>)）。迭代速度也值得关注：仅 2026 年 5 月中旬一次更新就在 72 小时内合并 14 个 PR，新增热记忆层、实时事实提取等生产级特性（来源：X 更新帖 (<https://x.com/garrytan/status/2054055071017538028>)）。Garry Tan 还公开表示，结合 OpenClaw/Hermes + GBrain 的方案，可以用 \$100/月达到原本 \$10,000/月方案的效果（来源：X 帖 (<https://x.com/garrytan/status/2055310116325560647>)）——这个数字是维护者的公开表述，不是我们的独立验证，但它说明了成本门槛在快速下降。

第二，它带着一套持续扩展的 skillpack。官方 README 当前写到 34 个 skills，`gbrain skillpack install` 还提供 25 个 curated skills 的安装路径。重点不是数字，而是设计方向：GBrain 把“怎么维护 brain”变成可安装、可检查、可升级的 Agent 工作流（来源：[GBRAIN_SKILLPACK](https://github.com/garrytan/gbrain/blob/master/docs/GBRAIN_SKILLPACK.md) (https://github.com/garrytan/gbrain/blob/master/docs/GBRAIN_SKILLPACK.md)）。

第三，它已经把跨模态、研究和运营型能力纳入同一框架。`book-mirror` 把一本书按章节做双栏分析，一边保留章节内容，一边映射到你的 brain 语境。翻译成跨境业务就是：让 Agent 带着你的产品、渠道、竞品、供应链判断去读行业报告和平台政策，再把有用判断回写到复盘系统。

公开讨论怎么帮助我们判断

这个判断不只来自我们的业务感受。[Repo Explainer](https://repo-explainer.com/garrytan/gbrain/) (<https://repo-explainer.com/garrytan/gbrain/>) 把 GBrain 放在 Agent memory operating layer 的位置；[ChooseAI](https://www.chooseai.net/news/3236/) (<https://www.chooseai.net/news/3236/>) 讨论了它相对 LLM Wiki 在 Postgres、pgvector、三层搜索上的工程推进；[XTrace](https://xtrace.ai/blog/every-tool-is-solving-a-different-memory-problem) (<https://xtrace.ai/blog/every-tool-is-solving-a-different-memory-problem>) 则把 Karpathy LLM Wiki 和 GBrain 放进 AI memory 工具版图里比较。我们做的，是把这些 AI 领域判断翻译成跨境电商的业务落地路径。

难点从工程转向理解

对非技术团队来说，GBrain 的好处是：很多工程层已经有开源项目承接。难点随之变化：你不一定要从零做数据库和 MCP，但你必须理解它为什么强调 page-first、compiled truth、timeline、typed links、source digest 和 governance。这本书要解决的正是这道理解鸿沟。

行动

用一句话写下你对 GBrain 的定位：它是笔记软件、搜索系统，还是 Agent 长期记忆层？如果你写不出来，这本书后面会帮你理清。

GBrain 是 LLM Wiki 方法论工程化最完整的开源答案——这一点前面两章已经讲清楚了。但在正式动手之前，你需要先学会它的基础语言。好消息是，GBrain 的概念体系虽然名词不少，只需要理解三层关系就能抓住全貌。

第三部分：理解 GBRAIN

04

基础概念：三层就够了

思考

22 个新术语听起来很吓人，但你只需要先理解 3 层关系。

先看一个对比。

示例

没有三层时：你问 Agent”厨房收纳盒北美站的广告预算该不该继续加？“Agent 搜了一堆包含”收纳盒”和”预算”的段落，拼出一个看起来合理的建议——但它不知道上次暂停预算是因为库存覆盖不足，不知道竞品 B 当时在低价冲量，也不知道团队后来复盘认为”那次暂停是正确的”。

有三层后：来源层保留了当时的渠道报告和竞品截图（证据）；知识层有一页厨房收纳盒产品页，Compiled Truth 写着”库存低于 21 天覆盖时暂停放量”，Timeline 记录了暂停-恢复-结果的完整轨迹；使用层让 Agent 通过 query 拿到这页，沿着关系找到关联的渠道页和竞品页，最终回答”上次暂停是正确的，当前库存已恢复，可以考虑重启，但注意竞品 B 仍在低价期”。

不需要先成为 AI 工程师。理解 GBrain 的起点是分清三层：来源层（材料从哪里来）、知识层（哪些判断被整理成稳定页面）、使用层（Agent 怎么检索和调用）。很多人卡住，是因为把三层混在一起。

来源层：保留证据，不急着写结论

术语	一句话解释	跨境业务例子
Document	一份完整原始材料	一份渠道报告、会议纪要、竞品观察文档
Raw source	不被 LLM 修改的原始来源	原始报表、截图、录音转写
Source digest	对来源材料的结构化摘要	一份竞品报告的要点、范围、时间、可信度
Chunk	为检索切出来的文本片段	长报告里被切出的 800 字片段

最重要的区分：source digest 保留来源证据，不能直接冒充业务结论。

知识层：把材料整理成人能读的页面

术语	一句话解释	跨境业务例子
Page	可命名、可链接、可维护的知识页	厨房收纳盒产品页、北美站渠道页
Wikilink	Markdown 内部链接	[[厨房收纳盒]] 指向产品页
Graph	对象之间的关系网	产品 → 渠道 → 决策
Typed link	有类型的关系	决策 -> 影响 -> 广告活动
Schema	页面结构和写作规则	type、product、channel、week、status

Compiled truth（当前最可信的综合判断）、resolver（把不同叫法对齐到同一对象）和 timeline（关键变化轨迹）也属于知识层，后续章节会展开。

最重要的区分：page 不是随手笔记，而是经过整理的、人和 Agent 都能理解的业务判断载体。

使用层：让 Agent 能检索、调用和维护

术语	一句话解释
Embedding	把文本变成可比较的向量——就像给每条业务记录贴一个隐形标签，让系统能判断哪些记录和你的
Index	帮系统快速找到内容的索引
Query	向知识库提问并综合回答
Sync / embed	把页面变化同步并更新检索索引
MCP	让 Agent 连接工具的标准接口——就像 USB 是让设备连接电脑的标准接口
Skill	可复用的任务方法

Eval（检查召回质量）和 lint（检查知识库健康）也属于使用层，在实战路线部分展开。

最重要的区分：embedding 找语义相似，graph 找业务关系——两者互补，不能互替。Obsidian 图谱读取 Markdown 文件和 wikilinks，GBrain 会把同一套 Markdown repo 同步进自己的检索和关系系统。

1. → Raw source（原始材料）
2. → Source digest（来源摘要）
3. → Formal page（正式知识页）
4. → Key Links / Schema / Timeline
5. → GBrain sync / embed / query
6. → Agent 在任务中调用

行动

挑你手边 5 条材料，按来源层、知识层、使用层重新标注：哪些只是证据，哪些应该写成页面，哪些问题要留给 query 检查。

05

Page-first 与双读者设计

思考

为什么不能把资料全丢给 embedding，还非要先写页面？

Page-first：先有人能理解的页面，再谈检索

Page-first 的核心是：跨境业务知识要先整理成可命名、可链接、可维护的页面，再让系统围绕页面做检索和关系。

这不是反对 chunk，也不是反对 embedding。它只是明确：业务知识的基本单位不应该是被机器切出来的片段，而应该是人和 Agent 都能理解的页面。

一个 page 可以承载当前结论、关键证据、相关对象、决策原因、后续结果、未关闭问题和时间线。一个 chunk 通常只能承载一段材料。它可能有用，但很难单独解释业务脉络。

1. → 先写稳定页面（结论、证据、链接、待跟进）
2. → 页面之间用 wikilink 连接
3. → GBrain sync / embed / query
4. → Agent 在任务中按页面和关系调用上下文

双读者设计：同一套知识，给人看，也给 Agent 用

GBrain 不是替代人类阅读，也不是让团队把知识交给黑箱。更好的结构是：同一套 Markdown brain repo，既给人看，也给 Agent 用。

1. → Markdown brain repo (唯一知识源)
2. → → Obsidian: 人类阅读、编辑、backlinks、图谱、复盘
3. → → GBrain: Agent 检索、typed links、timeline、query、MCP、维护

知识源只有一套，但使用方式有两种。人需要可读性、可编辑性、可追溯性。Agent 需要可检索、可连接、可调用、可维护。跨境电商不是纯工程系统——业务判断来自人的经验：产品定位、竞品直觉、渠道节奏、供应链风险。这些不能完全交给机器，也不能只留在人的脑子里。

示例

运营在 Obsidian 里补充一次厨房收纳盒的流量复盘判断：“北美站核心关键词转化稳定，但竞品 B 开始低价冲量，暂不加预算。”GBrain 同步后，Agent 下周做流量复盘时能读到这条判断，并在建议里说明来由。

行动

不要问“Obsidian 和 GBrain 选哪个”。先问：我的 Markdown 知识源能不能同时让人维护、让 Agent 调用？先为一个产品写 3 个页面：产品主页、一个渠道复盘页、一个决策页。

06

Obsidian 的位置：不是被替代，而是被更好利用

思考

有了 GBrain，还需要 Obsidian 吗？

需要，而且更需要把它放对位置。

先说最容易误会的一点：不要用 Obsidian 直接打开 GBrain 的工具仓库、数据库目录或 `.gbrain` 运行目录。Obsidian 应该打开的是你自己的 Markdown brain repo，也就是人类和 Agent 共同维护的知识源目录。

正确关系不是“Obsidian 打开 GBrain 底层数据库”，而是：

1. → Obsidian 打开 Markdown brain repo
2. → GBrain sync / embed 同一个 Markdown brain repo
3. → 两者共享知识源，各司其职

Karpathy 在 LLM Wiki 原文里用了一个好比喻：Obsidian 像 IDE，LLM 像程序员，wiki 像代码库（来源：[LLM Wiki \(https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f\)](https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f)）。放到 GBrain 语境下：Obsidian 是人类维护台，GBrain 是 Agent 记忆运行层。

Obsidian 的强项是人类体验：阅读 Markdown、编辑页面、看 backlinks、看局部图谱、做人工复盘、在页面之间自然跳转。在跨境业务里，它可以承担产品页的人工维护、复盘页的人工审核、竞品关系的手动链接、内容素材和关键词之间的关联、决策页的后续结果补充。

一个更安全的目录示例

```
seller-brain/                # Obsidian 打开这里; GBrain sync 这里
  products/
  channels/
  competitors/
  reviews/
  decisions/
  sources/digests/
  governance/

gbrain-tool-or-runtime/      # 不作为 Obsidian vault
  database/
  cache/
  runtime files/
```

如果团队把 GBrain 的运行目录当成 Obsidian vault，人类看到的不是清洁知识源，Agent 运行产生的索引和缓存也可能被误编辑。

行动

确认你的 Obsidian vault 指向的是 Markdown brain repo，而不是 GBrain 的工具仓库或数据仓库目录。

07

GBrain 的位置与理解门槛

思考

GBrain 在 AI 第二大脑里到底做什么？对非技术团队来说，最大的难点是什么？

GBrain 做什么：Agent 长期记忆层

GBrain 不替代 ERP，不替代飞书，不替代 Obsidian，也不替代业务判断。它让 Agent 能在任务中使用长期知识，重点处理六件事：

1. 把 Markdown 页面纳入可检索系统；
2. 用 Hybrid Search（混合检索——同时用关键词匹配和语义理解两种方式找内容，就像你既按产品名称搜，又按“上次为什么暂停”这种意思搜）；
3. 用 typed links 和 graph 表达对象关系；
4. 用 timeline 和 compiled truth 区分当前结论与历史变化；
5. 用 MCP / CLI 让 Agent 在真实 workflows 中调用；
6. 用 skillpack、recurring jobs 和维护流程，让 Agent 不只查知识，还能长期运营一套 brain。

GBrain 仍在快速演进。2026 年 5 月的更新引入了 hot memory（热记忆层——在同步时实时提取事实，让最新信息更快进入 Agent 可用范围）和多种新的 embedding 方案（来源：[X 更新帖 \(https://x.com/garrytan/status/2054055071017538028\)](https://x.com/garrytan/status/2054055071017538028)）。这些特性不需要你立刻理解，但它说明 GBrain 不是一个写好就放着的项目，而是一个生产环境中持续迭代的系统。

Agent 直接读文件夹也能工作，但文件多了以后找不到该看的页面、同义对象无法对齐、页面更新后索引不同步、关系只能靠模型猜、难以做 eval。GBrain 的意义，是把“读 Markdown”推进到“维护 Agent memory”。

难在哪里：不是安装，而是理解

很多人以为难点是工程化：数据库、pgvector、MCP、CLI。但对没有 AI 工程团队的跨境卖家来说，真正难的是理解 GBrain 的设计意图：

- 为什么要 page-first，为什么要保留 Markdown；
- 为什么不把所有原始资料都塞进去；
- 为什么需要 source digest，为什么正式页要有 compiled truth；
- 为什么时间线重要，为什么关系要写成 wikilink；
- 为什么 sync/embed 后还要 eval。

一旦理解这些，工程操作反而可以慢慢学、照文档做、让 Agent 辅助做。理解不到位，哪怕安装成功，也会把 GBrain 用成普通文件夹或搜索库。

示例

一个团队把所有报表、截图、会议记录都倒进知识库，看起来资料很多，但 Agent 仍然答不出“上次为什么暂停厨房收纳盒的北美站预算”。因为它没有页面、关系、决策和时间线。当 Agent 被要求复盘一个渠道，它不应该只搜索“渠道复盘”几个字，而应该能按产品、渠道、竞品、供应链约束、历史决策一起召回。

行动

回答一个问题：你的 Agent 能不能说出上次暂停某个渠道预算的原因？如果不能，缺的不是工具安装，而是知识结构。

理解 GBrain 的设计意图是第一步。下一个实操问题是：你的业务记忆应该围绕哪些对象组织？跨境电商的知识不是散装的——它天然围绕产品、渠道、竞品、广告活动、决策这些核心对象展开。搞清楚“记什么”，才能开始搭建。

第四部分：跨境业务知识怎么建模

08

跨境业务知识对象：第二大脑到底记什么

思考

跨境电商的 AI 第二大脑应该围绕哪些对象组织？

不要从文件夹开始，而要从业务对象开始。跨境电商的长期知识，通常围绕这些对象展开：

对象	例子	需要记录什么
Product	产品、变体、套装	定位、卖点、限制、复盘
Channel	Amazon、独立站、TikTok、线下渠道	流量结构、投放策略、渠道结果
Keyword / Query	搜索词、内容词、用户表达	意图、转化、素材匹配
Campaign	广告活动、内容活动、促销活动	目标、预算、动作、结果
Competitor	竞品、品牌、爆款链接	定价、内容、Review、动作
Supply Constraint	库存、交期、成本、供应商问题	对投放、定价、内容节奏的影响
Content Asset	图片、视频、A+、落地页、达人素材	使用场景、表现、复用关系
Decision	决策	背景、原因、动作、结果
Review	周复盘、月复盘、项目复盘	事实、分析、开放问题
SOP	流程	适用条件、步骤、例外

示例

一次厨房收纳盒的流量复盘不能只看广告活动，还要连接产品定位、内容素材、关键词、竞品 B 的降价动作和供应链约束。否则 Agent 只会给出”CPC 降了，可以加预算”这种不考虑约束的局部建议。

行动

打开你最近一次周会纪要，标注里面出现了哪些业务对象。你会发现大部分对话都在围绕上面这 10 类对象展开。

09

正式页、来源摘要页、治理页：不要把知识库写成文件夹

思考

页面应该分几层，才不会越写越乱？

一个常见错误，是把所有 Markdown 当成同一种东西：网页摘录、会议纪要、AI 总结、正式结论、治理规则全放在一起。短期很快，长期会让 Agent 分不清什么是证据、什么是观点、什么是已经确认的判断。

更稳的做法，是至少分三层：

页面层	作用	写入要求
来源摘要页	保存原始材料的压缩版	说明来源、时间、材料范围，不直接冒充结论
正式知识页	保存经过确认的综合判断	必须有 compiled truth、key links、evidence 或 timelin
治理页	规定命名、schema、resolver、写入边界	更新要谨慎，因为会影响后续 Agent 行为

Source digest 的价值是保存”这份来源材料提供了什么证据”，不应该直接变成业务结论。Formal page 保存的是经过人类确认、未来会复用的判断。Governance 规定系统怎么写——没有治理页，知识库越写越像个人笔记。

示例

一份竞品 B 的季度观察报告，先变成 source digest (“竞品 B 在 Q1 降价 15%，主打低价策略”)；产品负责人确认后，才把”竞品 B 正在用低价锚定用户预期，我们暂不跟价”写进正式竞品页；治理页规定：竞品判断必须带来源日期和观察范围。

行动

给你的 brain repo 加三类目录：`sources/`、`topics/`、`governance/`。

10

Compiled Truth + Timeline：既要当前结论，也要变化轨迹

思考

为什么页面里既要写当前判断，又要写时间线？Ch10 和 Ch11 一起回答一个问题：一页好的知识页面应该有什么结构。

跨境业务判断会变化。一个渠道今天要收缩，下个月可能要恢复；一个竞品今天只是短期降价，后面可能改变价格带。

如果页面只保存最新结论，Agent 看不到判断如何形成。如果只保存流水记录，Agent 又很难快速理解当前状态。所以页面至少需要两块：

Compiled Truth

- 当前最可信的判断是什么？
- 适用条件是什么？
- 下一次复核点是什么？

Timeline

- 2026-05-06：发生了什么。
- 2026-05-13：做了什么决策。
- 2026-05-20：后续结果怎样。

示例

厨房收纳盒的北美站渠道当前不继续放量——原因不是渠道失效，而是供应链交期仍超过 21 天。这写在 compiled truth。过去几次放量、暂停、恢复的时间和原因写进 timeline。

行动

把一个旧复盘页面改成 Compiled Truth + Timeline 两段，看看 Agent 是否更容易理解当前状态。

页面有了当前结论和时间线，但一页知识不能孤立存在——它需要和其他页面建立关系。Ch10 解决的是时间维度（当前结论 + 变化记录），下一章解决关系维度：链接、图谱、字段约定、别名对齐。

11

Key Links、Graph、Schema 和 Resolver

思考

为什么不能只靠模型自己理解业务关系？

因为业务关系太重要，不能全靠模型猜。

Key Links：关系要写出来。每个重要页面都应该有 Key Links，标明关联的产品、渠道、竞品、决策和广告活动。这些链接让 Obsidian 能看，让人能跳转，也让 GBrain 能抽取关系。

Graph：关系要有类型。typed links 可以让关系更清晰——决策 -> 影响 -> 广告活动、竞品 -> 压力 -> 产品定位、供应链约束 -> 限制 -> 渠道预算。

Schema：页面要有最低结构。frontmatter（页面开头的结构化信息——就像每份报告封面上的项目名称、日期、负责人）定义 type、product、channel、week、status 等字段。没有 schema，Agent 每次都要猜页面结构。

Resolver：同一对象要对齐。同一个产品可能有内部名、SKU、平台商品 ID、渠道链接、供应商型号。Resolver 把这些别名映射到一个 canonical id。

示例

厨房收纳盒在广告报告里叫 SKU-A001，在飞书里叫“厨收-基础款”，在供应链表里叫供应商型号 HG-201。没有 resolver，Agent 很容易把它拆成三个对象——业务记忆碎片化的起点。

一个最小的 RESOLVER.md 长这样：

```
| canonical_id | aliases | type |
| --- | --- | --- |
| product/kitchen-organizer-basic | SKU-A001, 厨收-基础款, HG-201 | product |
| product/kitchen-organizer-pro | SKU-A002, 厨收-升级款, HG-202 | product |
| channel/us-main | 北美站主账号, US-Store-1 | channel |
| competitor/brand-b | 竞品B, BrandB-US | competitor |
| campaign/us-kitchen-q2 | Q2北美厨收推广, Campaign-2026Q2-US | campaign |
```

当 Agent 收到查询“厨收-基础款上次为什么暂停预算”时，它先查 RESOLVER 把“厨收-基础款”对齐到 `product/kitchen-organizer-basic`，再沿着这个 canonical id 找到关联的决策页和广告活动页。没有 resolver，Agent 用“厨收-基础款”搜索可能找不到以 SKU-A001 命名的广告报告——同一个东西，三个名字，三份孤立的记忆。

行动

先写一个最小 `RESOLVER.md`，只处理你团队最常用的 10 个产品和渠道对象。

12

Skill + MCP 的能力边界

思考

既然有 Skill 和 MCP，为什么还需要 GBrain？

Skill 负责把方法固定下来——流量复盘怎么分析、竞品观察看哪些指标、哪些判断必须人工确认。
MCP 负责连接工具——读取表格、查询文档、调用浏览器、连接内部系统。

但 Skill 和 MCP 不天然沉淀长期判断。它们能让 Agent 做一次任务，却不能保证团队记住这次任务形成的业务结论。GBrain 补的是记忆层：过去判断是什么、证据是什么、后续结果怎样。这里说的记忆层有两种——团队的业务大脑和你个人的第二大脑。业务大脑是本书的主线，个人大脑在 Ch21-22 展开。

没有记忆层，AI 工作流像一次性脚本；有了记忆层，复盘才会积累。

示例

一个流量复盘 Skill 可以判断”北美站核心关键词是否值得继续投入”；GBrain 要记录”上次为什么保留它、后来结果怎样、这次是否仍适用”。

行动

把你已有的 Skill 分成两列：哪些是判断流程，哪些判断结果需要写入长期记忆？

Skill 和 MCP 解决执行，但判断结果谁来保管？下一章回答这个问题——GBrain 自带的 Skill 能力正是记忆层的维护机制。它不只是一个存储系统，还内建了一套保持记忆鲜活的工作流。

13

GBrain 的 Skill 能力与团队工具分工

思考

GBrain 自带这么多 skills，对卖家团队到底有什么用？飞书、Obsidian、GBrain、Codex、Skill、MCP 怎么放在一起？

GBrain Skill 能力：不是插件清单，而是 Agent 工作手册

不要把 GBrain 的 skills 理解成插件市场。GBrain 的 skillpack 更像 Agent 工作手册：什么时候该先查 brain，什么时候该摄取材料，什么时候该维护关系。官方 README 当前写到 34 个 skills，并由 `skills/RESOLVER.md` 组织路由（来源：[GBrain README \(https://github.com/garrytan/gbrain\)](https://github.com/garrytan/gbrain)）。Skillpack 文档也把它描述成生产级 Agent 使用 GBrain 的参考架构（来源：[GBRAIN_SKILLPACK.md \(https://github.com/garrytan/gbrain/blob/master/docs/GBRAIN_SKILLPACK.md\)](https://github.com/garrytan/gbrain/blob/master/docs/GBRAIN_SKILLPACK.md)）。

第一步不是”把所有 skills 都装上”，而是先理解五类能力层：

第一类：内容摄取。 `ingest`、`media-ingest`、`meeting-ingestion`、`voice-note-ingest` 等解决”材料怎么进入 brain”。对跨境团队意味着：行业报告、竞品长文、平台政策、会议纪要不要只留在聊天记录或网盘里。**适用条件：**团队有复盘节奏，但材料散在各处。

第二类：研究与综合。 `book-mirror`、`perplexity-research`、`strategic-reading` 等解决”Agent 怎么带着已有判断去读新材料”。读行业报告时，真正有价值的不是”这篇文章讲了什么”，而是”它会改变我们哪些业务判断”。**适用条件：**团队在看外部资料，但缺少映射回内部判断的流程。

第三类：Brain 运维。 `maintain`、`citation-fixer`、`query`、`skillpack-check`、`smoke-test` 等解决”知识库会不会越用越乱”。跨境业务第二大脑最怕两件事：错结论被反复召回，旧页面没人维护。**适用条件：**团队已经开始写 formal page，出现 stale、断链、重复对象的问题。

第四类：团队运营。 Skillpack 文档提到 recurring jobs、quiet hours、dream cycle、briefing 等操作纪律（来源：[GBRAIN_SKILLPACK.md \(https://github.com/garrytan/gbrain/blob/master/docs/GBRAIN_SKILLPACK.md\)](https://github.com/garrytan/gbrain/blob/master/docs/GBRAIN_SKILLPACK.md)）。跨境场景里可以更朴素：周复盘前自动准备历史判断，月度竞品分析前检查旧结论。**适用条**

件：从个人试点进入多人协作。

第五类：Skill 创建与检查。 `skillpack install / diff / check`、`skill-creator`、`skillify` 等解决”哪些经验值得固化成流程”。一个好 skill 不只是提示词，而是带输入、判断、边界、输出和验收标准的工作流。**适用条件：**某类任务反复做，但每次口径不一致。

最小组合：个人试点先用 `query` + `ingest` + `media-ingest`；复盘 MVP 加 `maintain` + `citation-fixer`；研究增强加 `book-mirror`；流程固化加 `skill-creator`。

团队工具分工

层	工具	负责什么
协作层	飞书 / 任务系统	会议、任务、审批、团队同步
人类知识层	Obsidian	阅读、编辑、backlinks、图谱、人工复盘
共享知识源	Markdown brain repo	人和 Agent 共用的页面源
Agent 记忆层	GBrain	query、typed links、timeline、MCP、维护循环
执行层	Codex / Skill / MCP	数据处理、业务判断、工具调用
自动化层	OpenClaw / 定时任务	周期流程、巡检、提醒

关键：飞书不是长期记忆，Obsidian 不是 Agent 服务端，GBrain 不是团队前台，Skill 不是知识库。

行动

列出你团队最常做的 3 类 AI 任务，每一类只回答一个问题：它现在缺的是材料摄取、研究综合、brain 运维、团队运营，还是 skill 固化？然后画出你团队的工具分层图。

第五部分：实战路线

14

从 20 条低风险知识样本开始

思考

第一批知识从哪里来？为什么不是把历史资料全搬进去？

从这一章开始，我们先聚焦业务大脑的搭建——团队共享的长期记忆系统。个人大脑的实操指南在 Ch22。

不要从搬运全部历史资料开始，也不要将复杂业务数据直接塞给 GBrain。GBrain 不会天然理解你的业务术语、渠道结构、产品命名和成本约束。第一步不是“喂数据”，而是先给它一批低风险、可读、脱敏、结构清楚的知识样本。

第一步：先做业务术语表

先写一页最小术语表，不超过 30 个词。从这些对象开始：产品、渠道、关键词、竞品、成本、供应链、决策字段、复盘状态。每个词只写三件事：它是什么，它有哪些常见别名，它会和哪些对象发生关系。

第二步：再整理 20 条低风险样本

建议结构：

- 5 条术语解释；
- 5 条流量复盘；
- 4 条竞品观察；
- 3 条约束记录；
- 3 条决策结果回看。

每条样本只回答四个问题：它涉及哪个对象？当时的判断是什么？它和哪些对象有关？后来有没有结果或待验证问题？如果结果还不知道，就写进 Open Threads。不要假装已经有结论。

第三步：最后才 sync / embed / query

等术语表和 20 条样本都能被人读懂，再接入 GBrain 做 sync / embed / query。第一轮不追求自动决策，只问 10 个验证问题：

- 厨房收纳盒关联过哪些渠道判断？
- 竞品 B 影响过哪些内容判断？
- 某次预算暂停和哪些约束有关？
- 哪些结论还没有结果回看？
- 哪些页面缺少来源或链接？

如果这些问题都答不出来，问题通常不在 GBrain，而在样本没有写清对象、关系和时间线。

行动

用 1 小时写一页业务术语表，再整理 20 条低风险样本。每条不超过 150 字，明确对象、判断、关系、时间线。

写完第一批样本后你可能会想：“我写的对不对？别人的样本长什么样？”欢迎通过「Zach的进化笔记」公众号加微信群直接交流，不需要任何付费门槛。如果你还想看更多卖家的实操案例和踩坑记录，知识星球里有持续的真实讨论——不是教程，是大家在落地过程中一起摸索。

15

最小 Markdown brain repo

思考

目录怎么搭，才能同时给人和 Agent 用？

第一版不要复杂。先搭一个能被 Obsidian 打开，也能被 GBrain import / sync 的 Markdown repo。

```
seller-brain/  
  README.md  
  schema.md  
  RESOLVER.md  
  products/  
  channels/  
  competitors/  
  campaigns/  
  content-assets/  
  reviews/  
  decisions/  
  sources/  
    raw/  
    digests/  
  governance/  
  inbox/  
  archive/
```

README 要写清用途：这个 repo 存储跨境业务的长期记忆，人在 Obsidian 里读和编辑，Agent 通过 GBrain 检索和调用。

schema.md 先定义 5 类页面就够：product（产品主页）、channel（渠道主页）、competitor（竞品主页）、business_review（业务复盘页）、decision（决策页）。

RESOLVER.md 先处理最常见的 10 个产品、渠道、竞品，不要一开始追求完整。

行动

建一个最小 repo，只写 5 个页面：1 个产品、1 个渠道、1 个竞品、1 个复盘、1 个决策。能被人读懂就算成功。

16

跨境业务复盘 MVP：先聚焦广告与流量

思考

第一个 MVP 应该选什么场景？

第一个场景建议选广告与流量复盘。原因很简单：它高频、有数据、有动作、有结果，也天然连接产品、关键词、内容、竞品和供应链。先跑通一个产品 + 一个主要渠道即可。

输入：流量或广告表现、产品当前状态、关键词、主要竞品变化、供应链或成本约束、历史复盘页、历史决策页。

处理：Codex / 脚本清洗数据 → Skill 按业务规则判断 → GBrain 召回历史判断 → Agent 生成复盘草稿 → 人类确认关键结论。

输出：复盘页（`reviews/YYYY-WW-traffic-review-*.md`）、决策页、飞书待办、Open Threads 列表。

最小判断问题：哪些流量动作有效？哪些动作受供应链或成本限制？哪些关键词值得继续观察？哪些竞品变化影响判断？哪些结论需要人类复核？

示例

Agent 发现厨房收纳盒的北美站转化变好，但 GBrain 召回历史页面显示：该产品库存覆盖不足 21 天，且上次放量导致交付风险上升。于是复盘建议不是“继续放量”，而是“保留核心流量，库存恢复后复核”。

行动

选一个产品和一个渠道，跑一次流量复盘 MVP。目标不是优化所有业务，而是验证第二大脑能不能召回旧判断。

17

第二场景：竞品分析

思考

除了广告流量，第二个适合进入 GBrain 的场景是什么？

第二个场景建议选竞品分析。主角是竞品，产品只是被影响对象。这样写，Agent 更容易判断哪些变化属于竞品页，哪些应该回流到产品页、内容页或决策页。

竞品分析关注：竞品价格带是否变化、核心卖点是否被模仿、内容表达有没有变化、Review 中的新问题是否影响我们的话术、上次竞品判断有没有结果回看。

竞品页不要写成”竞品资料收藏夹”。它至少要有四层：当前判断（这个竞品现在对我们意味着什么）、关键证据（价格、内容、Review、投放）、影响对象（关联哪些产品、渠道、内容素材）、Open Threads（哪些变化还需要继续观察）。

示例

竞品 B 突然在北美站低价上量，Agent 不能只建议降价。它应该先查竞品页、关联产品页、历史价格判断、内容素材效果和利润约束，再提出”跟价、强化差异化、调整内容表达、暂不响应”中的哪一种。

行动

为你最关注的一个竞品写一页 formal page，至少包含当前判断、关键证据、影响对象、后续观察项。

你现在有了两个实战场景的页面——流量复盘和竞品分析。但写在本地 Markdown 里的知识，Agent 还用不上。下一步是让这些知识真正进入 Agent 的工作流，从”人写了”变成”Agent 能查到、能用上”。

18

让知识进入 Agent workflow

思考

页面写好了，Agent 怎么真正用上？团队日常怎么跑？

sync / embed / query / MCP

GBrain 的接入流程可以理解成四步：

1. → sync：把 Markdown 页面同步到 GBrain
2. → embed：让更新后的页面进入检索索引
3. → query：让 Agent 或人查询长期记忆
4. → MCP：把 GBrain 接入实际 Agent workflow

用实际命令走一遍这四步（命令格式可能随 GBrain 版本变化，以官方文档为准——但核心四步逻辑是稳定的）：

```
# 1. 同步本地 Markdown 到 GBrain
gbrain sync ./seller-brain
# → Synced 47 pages (3 new, 2 updated, 42 unchanged)

# 2. 为新增/更新页面生成向量索引
gbrain embed --stale
# → Embedded 5 pages (3 new, 2 re-embedded)

# 3. 验证 Agent 能否召回关键判断
gbrain query "上次为什么暂停厨房收纳盒的北美站预算?" --no-expand
# → Found: decisions/2026-q1-kitchen-budget-pause
#   Compiled Truth: 因库存周期不确定性暂停, 等供应链确认后恢复
#   Key Links: product/kitchen-organizer-basic, channel/us-main
#   Timeline: 2026-03-15 暂停 → 2026-04-02 供应链确认 → 2026-04-05 恢复

# 4. 通过 MCP 让 Agent 在任务中调用
# Claude Code / Cursor 中 GBrain MCP 已自动加载, Agent 可直接查询
```

Obsidian 编辑的是 Markdown 源，GBrain 需要知道哪些页面更新了（sync）。页面更新后，向量索引也要更新（embed），否则人看到新页面，Agent 找到的可能还是旧内容。query 是检查 GBrain 是否真的能召回判断的入口——不要只问泛问题，要带对象和任务。MCP 让 Agent 在任务中调用 GBrain，而不是人手动复制页面。

团队协作链路

一个实际可用的日常流程：

1. → 飞书发起复盘任务
2. → Codex 读取报表和资料
3. → Skill 执行业务判断流程
4. → GBrain 召回历史判断
5. → Agent 生成复盘草稿
6. → 人类审核关键结论
7. → Markdown brain repo 写入正式页
8. → Obsidian 人工阅读和补链接
9. → 飞书回流待办

飞书回流不要太重——只承接本周确认动作、需要审批的动作、下周观察项、负责人、截止时间和对应知识页标题。不要把整篇知识页塞回飞书。

行动

每次编辑后至少跑 3 个 query，确认 Agent 找到的是最新判断。然后设计一个不超过 6 个字段的飞书回流模板。

19

轻量 eval：怎么知道第二大脑真的有用

思考

怎么判断 GBrain 不是写得热闹，而是真的能用？

要做 eval。第一版不需要复杂，先用 10 个问题检查能不能召回、解释、引用、更新。

建议问题：

1. 厨房收纳盒最近一次暂停渠道预算的原因是什么？
2. 哪些复盘结论受到供应链约束影响？
3. 哪个竞品最近改变了我们的产品判断？
4. 哪些内容素材被证明有效？
5. 哪些决策没有后续结果？
6. 哪些页面缺 Key Links？
7. 哪些 Open Threads 超过两周未关闭？
8. 哪些结论需要人工复核？
9. 哪些页面的 compiled truth 和 timeline 冲突？
10. 哪些模型推断还没有来源支持？

记录错误类型：没搜到、搜到但对象错、结论过期、缺证据、把推断当事实、引用不完整、没有提出复核问题。

行动

建一个 `evals/business-review-questions.md`，每周用同一批问题抽查。

从手动到半自动

每周手动跑 10 个问题是起点，但人会忘、会跳过。把 eval 变成一个简单脚本，每周花 5 分钟确认大脑没退化：

```
#!/bin/bash
# evals/run-eval.sh - 最小 eval 脚本
questions=(
    "厨房收纳盒最近一次暂停渠道预算的原因是什么？"
    "哪些复盘结论受到供应链约束影响？"
    "哪个竞品最近改变了我们的产品判断？"
)
for q in "${questions[@]}; do
    echo "--- Q: $q"
    gbrain query "$q" --no-expand 2>&1 | head -5
    echo ""
done
```

跑完后人工看一遍输出：能召回的打 ，召回错误的标记错误类型。这不是工业级 eval 框架，而是“最小验证习惯”——目标是让 eval 从“知道应该做”变成“每周真的做了”。

跑完第一轮 eval 后你会发现：10 个问题不够用，而且不同业务的问题差别很大。我自己的 eval 问题库也在持续扩展。如果你也在做这件事，「Zach 的进化笔记」知识星球里有更多卖家的真实业务问题——比起一个人闭门造车，更多人的真实场景能让 eval 更有代表性。

20

企业级使用 GBrain：从个人 brain 到团队 Agent 记忆层

思考

一个人能用 GBrain，团队怎么用才不乱？

企业级不是把个人 brain 直接共享给所有人，而是先把个人可用的 Agent Brain，逐步升级成团队可查询、可审计、可维护的 Agent 记忆层。

企业级价值

价值	解决什么问题	跨境业务里是什么
共享业务记忆	判断不只留在个人聊天和脑子里	新负责人接手时能查到过去复盘
决策可追溯	知道为什么做过某个动作	预算、内容、渠道判断有来源和时间线
新成员 / 新 Agent onboarding	新人和新 Agent 不从零开始	先读产品页、竞品页、复盘页、决策页
跨工具一致性	不同入口看同一套知识	飞书、Obsidian、Codex、Agent 查同一套 fc
权限和审计	知道谁能读、谁能写、谁调用过	读写分权，关键请求保留 request log

落地路径：五步走

第一步：个人试点。 先用本地 Markdown brain repo 跑通 20 条低风险样本、10 个 eval 问题和一个复盘 MVP。只验证 Agent 能不能召回对象、关系、判断和时间线。

第二步：小团队只读查询。 让运营负责人、业务负责人或外部 Agent 先只读查询。只读阶段暴露 formal page 和 eval 结果，不开放随意写入。目标是建立信任。

第三步：负责人写入。 普通成员提交来源摘要或复盘草稿，负责人确认后再写入 formal page。减少幻觉写入和口径污染。

第四步：定期维护和 eval。 每周固定跑 query / eval / maintain，检查 stale 页面、断链、缺 citation、Open Threads 超期和命名冲突。

第五步：企业治理。 明确 schema、resolver、读写权限、审计保留、敏感信息边界和退出机制。企业级不是功能更多，而是出错时能追责、能修复、能降级。

Remote MCP：从个人用到团队用

给决策者的三个判断

什么时候从个人用切换到团队用？ 当超过一个人需要查询业务大脑时。标志通常是：运营同事开始问你“上次那个判断在哪里”，或者你发现自己在复制粘贴 GBrain 的查询结果给别人。GBrain 已经在 multiplayer 场景中被使用——Garry Tan 确认他的 7 人团队正在用 GBrain 的多人模式，并且支持 federated repos（联邦仓库），可以让个人仓库和工作仓库并存（来源：[X 回复 \(https://x.com/garrytan/status/2055884666264269177\)](https://x.com/garrytan/status/2055884666264269177)）。

持续成本是什么？ 技术成本很低——GBrain 自带团队接入能力，不需要额外采购。真正的成本是管理成本：谁负责维护页面质量，谁审核写入，谁处理权限申请。

需要给技术人员什么要求？ 三条底线：每个人和每个工具用独立的访问凭据（不共享密码）、默认只给查看权限（写入需要审批）、人员变动时及时撤销权限。

给执行者的配置参考

本地单人使用时，`gbrain serve` 通过本地直连方式（stdio）给 Claude Code、Cursor 等工具调用。团队或远程工具接入时，官方 README 提到 `gbrain serve --http` 可以启动 Remote MCP，带 OAuth 2.1、admin dashboard、scopes、request log 等能力（来源：[GBrain README \(https://github.com/garrytan/gbrain\)](https://github.com/garrytan/gbrain)）。

最小操作逻辑：

```
gbrain serve --http --port 3131
open http://localhost:3131/admin
```

然后在 admin dashboard 里注册 client、选择 scopes、保存凭据。不要把“开一个 MCP key 给大家用”理解成群共享密码。更安全的做法：

- 按成员或工具注册独立 client / token；
- 默认只给 `read`；

- `write` 只给负责维护 formal page 的角色；
- `admin` 只给系统维护者；
- 成员离开、工具废弃时及时 revoke；
- 不把 token 写进 Markdown、飞书文档或公开仓库。

如果使用旧式 bearer token，GBrain Security 文档给出 `gbrain auth create / list / revoke / test` 的管理方式，token 明文只在创建时显示一次（来源：[SECURITY.md](https://github.com/garrytan/gbrain/blob/master/SECURITY.md) (<https://github.com/garrytan/gbrain/blob/master/SECURITY.md>)）。

权限设计：默认 read，谨慎 write

角色	建议权限	原因
普通运营成员	read	查历史、查复盘、查术语
业务负责人	read + limited write	确认复盘和决策后写入
Agent 执行工具	read 优先，write 需单独批准	防止自动写入污染正式页
系统维护者	admin	管理 client、token、schema、维护任务

安全边界

公开 Security 文档提醒：不要开放未认证的 OAuth client registration，否则发现服务器地址的人可能注册 client、拿到 token、访问 brain 数据。文档还建议限制 scopes、记录 token 签发、对注册和 token 端点做 rate limit（来源：[SECURITY.md](https://github.com/garrytan/gbrain/blob/master/SECURITY.md) (<https://github.com/garrytan/gbrain/blob/master/SECURITY.md>)）。

生产环境至少保留四个默认动作：审计（保留 request log）、限流（IP / token 维度 rate limit）、脱敏（request log 默认保留 redacted params）、撤销（每个成员和工具都有独立 token，可单独 revoke）。

行动

写一页 `ACCESS_POLICY.md` 草案，只定义四件事：谁能读、谁能写、谁能管理、出问题如何 revoke。

企业级解决的是团队协作问题——多人怎么共享一个业务大脑。但还有一个更个人的问题：你和 Agent 之间的长期关系，需要另一个大脑来承载。前言中的第五条判断在这里展开。

21

为什么你需要两个大脑：业务大脑与个人大脑

思考

公司的业务判断和个人的 AI 增强，应该放在同一个 brain 里吗？

不应该。业务大脑和个人大脑解决的是两个不同层面的问题，混在一起会带来权限混乱、噪声干扰，甚至离职时无法干净分离。

一个 brain 为什么不够用

业务 brain 是团队资产。它存放的是产品、竞品、复盘、决策——这些信息属于公司，服务公司所有 Agent 和团队成员。Ch20 讲的企业级接入（Remote MCP、OAuth、权限分层）就是为这一层设计的。

个人 brain 是你自己的 AI 增强。它存放的是你的判断习惯、工作偏好、过往认知积累和跨模型的记忆——这些信息属于你个人，不管你在哪家公司、用哪个模型，它都应该跟着你走。

如果把两者混在一起：团队成员能看到你的个人偏好，你的个人笔记污染业务 brain 的信噪比，换工作时你要么带走公司数据、要么丢掉自己的积累。分开是更干净的架构。

这不只是我们的理论推导——GBrain 原生支持这种分离。Garry Tan 确认 GBrain 支持 federated repos（联邦仓库），可以让个人仓库和工作仓库在同一套系统中并存，也可以通过多个

`GBRAIN_HOME` 实例实现完全隔离（来源：[X 回复 \(https://x.com/garrytan/status/2055884666264269177\)](https://x.com/garrytan/status/2055884666264269177)）。换句话说，双脑架构不是对 GBrain 的额外改造，而是它设计中已经考虑到的使用方式。

真实痛点：跨模型记忆散乱

你可能同时在用 Claude、ChatGPT、Gemini、Cursor——每个模型只记得跟它自己聊过的内容。

上周你在 Claude 里花了两个小时梳理北美站竞品格局，形成了一系列判断。这周你换到 ChatGPT 问同一个问题，它完全不知道你上周的结论。你不得不从头解释。不是因为 ChatGPT 不够好，而是你的判断被锁在了 Claude 的对话历史里。

个人 brain 解决的就是这件事：把你跨模型的判断沉淀到一个 Agent 都能读的外部知识层。不管你今天用哪个模型，它都能读到你之前的结论、你的工作偏好、你的决策习惯。

附带一个好处：当模型账号出现迁移、降级甚至封号的情况时，你积累的上下文和记忆不会随之消失——因为它们从一开始就没有存在模型里。

GBRAIN_HOME：一个环境变量实现双实例隔离

GBrain 通过 `GBRAIN_HOME` 环境变量指向不同的数据目录。每个实例有独立的 PGLite（GBrain 内置的轻量数据库，你不需要单独安装或管理）数据库、embedding 索引和 page 体系。你不需要安装两套软件，只需要两个目录：

```
# 业务大脑
export GBRAIN_HOME=~/.brain-business
gbrain stats

# 个人大脑
export GBRAIN_HOME=~/.brain-personal
gbrain stats
```

可以为每个实例写一个简单的 shell 包装脚本，避免每次手动设置环境变量：

```
#!/bin/bash
# ~/.bin/gbrain-personal
export GBRAIN_HOME=~/.brain-personal
exec gbrain "$@"
```

两个实例各自独立运行 `sync`、`embed`、`query`，互不干扰。

双脑分工表

维度	业务大脑	个人大脑
归属	团队 / 公司	你自己
接入方式	Remote MCP + OAuth	本地 studio
典型内容	产品页、竞品页、复盘、决策、术语表	个人画像、工作偏好、跨模型记忆、学习笔记
权限管理	read / write / admin 分层	不需要权限管理
团队可见性	所有授权成员和 Agent 可查	仅你自己
换公司时	留在公司	跟你走
主要价值	组织记忆不随人员流动丢失	个人 AI 增强不被锁在某个模型里

行动

列出你目前跨模型使用时最常重复解释给 AI 的三件事——你的角色、你的行业背景、你的工具链偏好。这就是个人 brain 的第一批内容。

22

个人大脑实操指南：从目录到跨模型迁移

思考

个人大脑的目录怎么建、画像页怎么写、怎么让所有模型都读到？

个人 brain 的目录结构

建议从三个目录开始：

```
personal-brain/
├─ profile/           ← 个人画像、工作偏好、沟通风格
├─ knowledge/         ← 跨模型沉淀的判断和学习笔记
├─ decisions/         ← 重要个人决策和复盘
├─ schema.md          ← 页面类型和字段定义
└─ RESOLVER.md        ← 命名和写入规则
```

和 Ch15 的业务 brain 做个对比：

维度	业务 BRAIN 目录	个人 BRAIN 目录
核心目录	topics/ 、 sources/ 、 governance/	profile/ 、 knowledge/ 、 decisions/
schema 重点	产品、竞品、复盘、决策	个人画像、偏好、学习笔记
页面数量	几十到几百页	起步 5-10 页
维护频率	团队定期维护	个人随时更新

个人画像页的 schema 设计

画像页是个人 brain 最核心的页面。它告诉 Agent”你是谁”，让每次对话不从零开始。

建议 frontmatter：

```

---
type: personal_profile
owner: your-name
status: active
last_updated: 2026-05-15
tags: [profile, core]
---

```

正文结构建议分五个区块：

1. **角色与背景**：你做什么、在什么行业、什么阶段
2. **工作偏好**：沟通风格（直接还是铺垫）、决策习惯（数据驱动还是直觉优先）、常用工具链
3. **当前关注领域**：最近 3 个月的工作重心
4. **AI 使用习惯**：你期望 AI 怎么跟你协作——先给结论还是先分析？需要多方案还是直接推荐？
5. **不需要重复解释的上下文**：你的行业常识、团队术语、过往已经定下的判断

画像页怎么写才能让 Agent 真正用上

画像页不是简历，是”你希望 AI 在下次对话开始前就知道的事”。

写的时候区分两类信息：

- **事实**：你的角色、行业、工具链、团队规模。这些不常变。
- **偏好**：你喜欢什么样的输出格式、什么时候需要详细分析、什么时候只要结论。这些会随着你和 AI 的协作经验演变。

一个虚构的跨境电商运营负责人画像页示例：

个人画像：运营负责人

角色与背景

跨境电商公司运营负责人，负责北美站和欧洲站。团队 5 人。

主要品类：家居收纳。3 条产品线，年营收 500 万美元量级。

工作偏好

- 沟通风格：直接给结论，再展开分析。不需要铺垫和背景介绍。
- 决策习惯：数据优先，但最终靠业务判断拍板，不迷信模型。
- 工具链：飞书（团队协作）+ Obsidian（个人笔记）+ Claude Code（AI 主力）+ ChatGPT（补充验证）。

当前关注

- 北美站厨房收纳盒的利润率下降，需要判断是竞品低价冲量还是品类天花板。
- 欧洲站合规成本上升，评估是否缩减 SKU。

AI 使用习惯

- 分析报告先给结论和建议，数据支撑放后面。
- 提供多方案时必须给出每个方案的优缺点和推荐理由。
- 不需要解释基础亚马逊运营概念（ACOS、TACOS、BSR 等）。

不需要重复解释的上下文

- "库存覆盖不足 21 天时暂停放量"是团队已确认的规则。
- 供应商交期通常 45-60 天，海运 25-35 天。
- 广告预算按月度分配，不做日维度微调。

画像页应该定期更新。建议每月回顾一次：关注领域变了吗？偏好变了吗？有没有新的“不需要重复解释的上下文”？

跨模型接入：让 Claude、ChatGPT、Cursor 都读同一个 brain

个人 brain 的核心价值在于跨模型可用。不同工具的接入方式不同：

Claude Code（原生 MCP）：直接在 MCP 配置中指定个人 brain 的 `GBRAIN_HOME`，`gbrain serve` 通过 stdio 接入。Agent 在对话中自动查询个人 brain。

其他支持 MCP 的工具（Cursor、Windsurf 等）：用 `gbrain serve --http --port 3132` 启动本地 HTTP 服务。注意：个人 brain 在本地运行，不需要走团队的 Remote MCP 和 OAuth——它只服务你自己。

不支持 MCP 的模型（ChatGPT 网页版、Gemini 等）：降级方案。用 `gbrain query "我的画像和工作偏好" --no-expand` 导出相关页面内容，粘贴到对话开头作为上下文。不如 MCP 自动化，但至少不需要每次手动重复。

关键原则：个人 brain 永远在本地。它的数据不走公网、不需要 OAuth、不需要权限管理。你的画像和偏好只有你自己能读。

跨模型迁移清单

如果你已经在某些模型里积累了记忆和偏好，可以迁移到个人 brain：

从 ChatGPT Memory 迁移： 1. 在 ChatGPT 设置中导出 Memory 内容（Settings → Personalization → Memory → Manage → Export） 2. 整理成 Markdown page：事实类信息写入画像页，判断类信息写入 `knowledge/` 目录 3. `gbrain import` 导入个人 brain

从 Claude Projects 迁移： 1. 复制 Project Instructions 的内容 2. 角色、偏好、工具链部分写入画像页；业务上下文部分写入 `knowledge/` 目录 3. `gbrain import` 导入个人 brain

从 Cursor Rules 迁移： 1. 读取 `.cursorrules` 或 `.cursor/rules` 文件 2. 编码偏好、项目约定写入 `knowledge/coding-preferences.md` 3. `gbrain import` 导入个人 brain

迁移不是一次性搬家。先迁移最常重复解释的 3-5 条信息，跑通查询验证后再逐步扩展。

双脑协作：个人判断何时升级为业务判断

两个 brain 不需要自动同步，靠你的判断决定什么该流动：

从个人到业务： 你在个人 brain 里记录了一个竞品观察——连续三周跟踪某个竞品的定价策略变化。当你判断这个观察对团队有价值时，整理成正式的竞品分析页，经团队确认后写入业务 brain 的 `topics/competitors/` 目录。

从业务到个人： 团队在业务 brain 里确认了“库存覆盖不足 21 天时暂停放量”这条规则。你在个人 brain 的画像页中引用这条规则——这样即使你在非工作场景使用 AI，它也知道你团队的关键业务约束。

核心原则：业务 brain 的内容需要团队共识才能写入（Ch20 的五步走），个人 brain 的内容由你自己决定。两者之间的流动是手动的、经过判断的，不是自动同步。

行动

建一个 `personal-brain/` 目录，写第一页个人画像。先写三段：你的角色、你的工具链、你最不想每次重复解释的上下文。不需要完美——先有，再迭代。

第六部分：风险与下一步

23

什么不该进入 GBrain

思考

GBrain 会不会变成新的资料垃圾桶？

会，如果没有边界。

适合进入 GBrain 的内容： 经确认的业务判断、复盘结论、决策原因、后续结果、关键关系、来源摘要、治理规则、可复用 SOP。

不适合直接进入 GBrain 的内容： 未清洗原始 CSV、大量截图、无结论聊天记录、未确认模型推断、敏感凭证、临时脑暴、可从系统实时查询的明细数据。

GBrain 不是数据仓库，也不是资料收纳箱。它应该沉淀”判断、关系、来源摘要、复盘、决策”。

示例

一份北美站的渠道原始报表不需要全量写入 GBrain。应该先由脚本或 Agent 处理，再把”本周关键异常、判断、动作、后续观察”写进复盘页。

行动

给每条材料打标签：raw data、source digest、formal knowledge。只有后两类考虑进入 GBrain。

24

风险：幻觉写入、知识污染、维护成本、过度自动化

思考

GBrain 可能带来什么新风险？

AI 第二大脑不是越大越好。真正的风险在于，它会让错误判断变得更持久。

幻觉写入：模型推断没有来源，却被写成正式结论。解决办法：关键判断必须区分 Fact、Analysis、Seller Fit。

知识污染：重复页面、旧结论、冲突命名、断链页面越来越多。解决办法：定期维护、合并、标记 stale、跑 eval。

维护成本：页面结构太复杂，团队写不动。解决办法：第一版 schema 要轻，只保留真正会复用的字段。

过度自动化：Agent 直接把建议当动作执行。解决办法：关键业务动作必须有人类确认，特别是预算、价格、库存、供应链和合规相关动作。

行动

写一页 **BOUNDARIES.md**，明确哪些内容可以自动写，哪些只能建议，哪些必须人工复核。

25

给不同团队的落地建议

思考

不同阶段的跨境团队，应该从哪里开始？

非技术卖家老板

你不需要先懂数据库和 MCP。先判断三件事：团队是否已经有稳定复盘、是否有重复出现的业务判断、是否希望 Agent 在下次任务前读到历史。如果这三件事都成立，AI 第二大脑值得开始做最小版。

懂一点 AI 工具的运营负责人

先搭流程，不追工具完整度：一个复盘场景、一个 Markdown repo、一个 Obsidian 编辑入口、一个 GBrain 查询入口、一个 Skill 判断流程、一个飞书回流模板。

准备搭内部 AI 工作流的执行者

先把基础工程做稳：schema、resolver、source digest、formal page、sync/embed、query、eval、写入边界。

个人用户 / 独立卖家

如果你不是团队，而是一个人在做跨境电商，或者你想在团队工作之外建自己的 AI 增强：先建个人 brain，写画像页和工作偏好，把跨模型重复解释变成 Agent 能读的页面。不需要 Remote MCP，不需要 OAuth，一个本地 `gbrain serve` 就够。等你觉得某些判断对团队也有用时，再考虑写入业务 brain。

行动

按你的角色选一个最小动作。老板定场景，负责人定流程，执行者定结构，个人用户写画像页。

26

结语：从 Skill 工具箱到 AI Native 第二大脑

思考

读完以后，应该带走什么判断？

跨境电商的 AI 化会经历三个阶段：

第一阶段，是 prompt 和单次问答——让 AI 帮我做一点事。

第二阶段，是 Skill 和 MCP——让 Agent 按流程做事、连接工具做事。

第三阶段，是 AI Native 第二大脑——让 Agent 带着组织的长期判断做事。

GBrain 的价值，在第三阶段才真正出现。它不替代 Obsidian，也不替代人类判断，而是让跨境电商团队的产品、流量、竞品、供应链和复盘判断，既能被人读懂、编辑、追溯，也能被 Agent 检索、调用、维护。

但三个阶段只解决了”组织”的问题。还有一个更长远的问题：你和 Agent 之间的关系。

今天你每开一个新对话，Agent 都像一个刚入职的新同事——聪明，但对你一无所知。你不断重复你的背景、偏好、过往判断，它不断遗忘。这不是某个模型的缺陷，而是”记忆被锁在对话里”的结构性问题。

个人大脑改变的就是这个结构。当 Agent 能够持续读取你的画像、偏好、决策历史和长期积累的认知，你们之间才可能建立真正的长期协作——不是每次从头开始的问答，而是基于持续理解的搭档关系。业务大脑服务组织，个人大脑服务你自己。两层记忆，才是完整的 AI Native 第二大脑。

如果你今天只记住一句话，就记住这句：

跨境电商的 AI 第二大脑，不是资料越多越好，而是每一次业务判断都能被下一次 Agent 正确召回。

从一个跨境业务复盘页面开始。只要下一次 Agent 因为这页少犯一次旧错误，这套系统就开始有价值。

这本书是初稿。很多判断还在验证，很多做法还在迭代。我自己也在用这套方法搭建和维护跨境电商的 AI 第二大脑，走到今天的判断不一定全对，但至少是真实踩过坑之后的总结。

如果你尝试落地后有反馈、有问题、或者想看看别人怎么做的——欢迎通过「Zach的进化笔记」公众号加入微信交流群，直接聊，不需要任何付费门槛。如果你还想看更多实操过程的记录和讨论，知识星球是另一个选择：不是课程，是一群正在落地的人的持续讨论。

这本书会一直开源。后续的迭代和修正会同步更新到 GitHub 仓库，你随时可以拿到最新版。

扫码交流

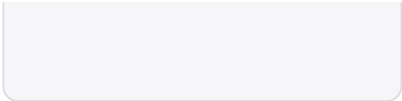


公众号：Zach的进化笔记

回复“第二大脑”加读者群
免费交流，有问题直接聊



知识星球：Zach的跨境AI资产
库



深入交流 AI 落地到企业的卡点，
沉淀未来的 AI 资产

附录 A：术语表

术语	简明解释	跨境业务例子
Document	一份完整材料	渠道报告、会议纪要、竞品观察
Page	可命名知识单元	产品页、复盘页、决策页
Chunk	检索片段	从长报告切出的段落
Embedding	文本向量	找语义相似的业务问题
Index	检索索引	按产品、渠道、向量快速找
Wikilink	Markdown 内部链接	[[厨房收纳盒]]
Graph	关系网	产品 → 渠道 → 决策
Typed links	有类型的关系	决策 -> 影响 -> 广告活动
Schema	页面结构约定	frontmatter 字段
Resolver	名称解析	SKU、平台 ID、内部名对齐
Source digest	来源摘要页	一份竞品报告的证据摘要
Formal page	正式知识页	经确认的产品复盘结论
Compiled Truth	当前可信结论	当前渠道策略
Timeline	关键变化记录	每次调整和结果
MCP	Agent 工具接入协议	Agent 查询 GBrain
Skill	业务判断流程	流量复盘规则
Eval	检查问题集	“为什么上次暂停预算？”

附录 B：样例页面模板


```

---
type: business_review
product: 厨房收纳盒-基础款
channel: 北美站-搜索广告
week: 2026-W20
owner: growth
status: active
source_type: codex_analysis
review_required: true
---

```

厨房收纳盒-基础款 跨境业务复盘 2026-W20

Compiled Truth

- 当前核心渠道维持预算，不继续放量。
- 主要原因不是流量无效，而是供应链约束仍未解除。
- 下次复核条件：库存覆盖超过 21 天，且转化率保持稳定。

Key Links

- Product: [厨房收纳盒-基础款]
- Channel: [北美站-搜索广告]
- Campaign: [核心搜索广告-Q2]
- Competitor: [竞品B-低价收纳]
- Decision: [Decision-2026-05-13-维持预算]

Fact

- 本周核心渠道转化率保持稳定。
- 供应链交付周期仍高于目标区间。
- 竞品 B 出现短期降价。

Analysis

- 继续放量可能增加交付风险。
- 竞品降价暂不构成长期价格带变化。

Seller Fit

- 适用于当前库存和交付受限的产品。
- 若供应链恢复，需要重新评估渠道预算。

Open Threads

- 下周复核供应链覆盖。
- 观察竞品 B 降价是否持续。

Timeline

- 2026-05-06：渠道表现恢复，但供应链仍受限。
- 2026-05-13：决定维持预算，不继续放量。

附录 C：七步上手清单

步骤	做什么	参考命令 / 操作
1	写一页最小业务术语表	在 <code>seller-brain/</code> 下新建 <code>glossary.md</code>
2	整理 20 条低风险知识样本	每条不超过 150 字，明确对象、判断、关系、时间线
3	建最小 Markdown brain repo	按 Chapter 15 的目录模板建文件夹
4	写 <code>schema.md</code> 和 <code>RESOLVER.md</code>	参考附录 B 的 frontmatter 字段
5	用 Obsidian 打开 repo，确认人能阅读和编辑	Obsidian → Open folder as vault
6	接入 GBrain，执行 sync / embed / query	<code>gbrain import ./seller-brain</code> → <code>gbrain embed --st</code>
7	建 10 个 eval 问题，每周抽查	先人工回答，再让 GBrain 回答，对比差距；多人接入前写

附录 D：来源与参考资料

本书只在公开稿中标注可公开访问的来源。内部搭建记录和讨论只作为写作输入，不作为公开引用列出。

公开来源：

- [Karpathy: LLM Wiki](https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f) (<https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>)：用于确认 LLM Wiki 的核心思想、三层架构、ingest/query/lint、index.md、log.md、Obsidian 与 LLM 的关系。
- [garrytan/gbrain 官方仓库](https://github.com/garrytan/gbrain) (<https://github.com/garrytan/gbrain>)：用于确认项目定位、核心模块和公开能力边界。
- [GBrain README](https://github.com/garrytan/gbrain/blob/master/README.md) (<https://github.com/garrytan/gbrain/blob/master/README.md>)：用于确认 Hybrid Search、self-wiring graph、structured timeline、MCP、34 skills、skillpack、book-mirror、media-ingest、brain-pdf、X-to-Brain、Voice-to-Brain 等公开说明。
- [GBrain Skillpack docs](https://github.com/garrytan/gbrain/blob/master/docs/GBRAIN_SKILLPACK.md) (https://github.com/garrytan/gbrain/blob/master/docs/GBRAIN_SKILLPACK.md)：用于确认 Brain-Agent Loop、Content & Media Ingestion、20+ recurring jobs、X-to-Brain、Voice-to-Brain、Upgrades & Auto-Update 等框架化能力。
- [GBrain Security](https://github.com/garrytan/gbrain/blob/master/SECURITY.md) (<https://github.com/garrytan/gbrain/blob/master/SECURITY.md>)：用于确认 Remote MCP 的 token 管理、OAuth client registration 风险、rate limit、audit log 和 redacted params。
- [Garry Tan 关于 GBrain total recall 的 X 推文](https://x.com/garrytan/status/2042497872114090069) (<https://x.com/garrytan/status/2042497872114090069>)：用于确认他公开把 GBrain 描述为 OpenClaw / Hermes Agent 的长期记忆。
- [Garry Tan 关于 GBrain voice/context 的 X 推文](https://x.com/garrytan/status/2043103243732226496) (<https://x.com/garrytan/status/2043103243732226496>)：用于确认 GBrain 进入 voice/context 场景后的公开讨论。
- [X trending: Garry Tan Open-Sources GBrain for AI Agent Memory](https://x.com/i/trending/2042548874871513262) (<https://x.com/i/trending/2042548874871513262>)：辅助来源。
- [Y Combinator: Garry Tan](https://www.ycombinator.com/people/garry-tan) (<https://www.ycombinator.com/people/garry-tan>)：用于确认 Garry Tan 的公开身份。
- [Repo Explainer 对 GBrain 的解读](https://repo-explainer.com/garrytan/gbrain/) (<https://repo-explainer.com/garrytan/gbrain/>)：用于参考 agent-maintained memory 与普通 Obsidian + embedding 的区别。
- [ChooseAI 对 GBrain 的中文介绍](https://www.chooseai.net/news/3236/) (<https://www.chooseai.net/news/3236/>)：用于参考 LLM Wiki 到工程化检索层的延伸。

- XTrace 关于 AI memory 新范式的文章 (<https://xtrace.ai/blog/every-tool-is-solving-a-different-memory-problem>): 用于参考 Karpathy LLM Wiki 与 GBrain 所在的更大讨论。
- Garry Tan: GBrain 主介绍线程 (2026/5/16) (<https://x.com/garrytan/status/2055670533451366479>): 用于确认 GBrain 定性为”knowledge system, not RAG in a box”、8 层记忆架构、Personal AI、MIT 开源。
- Garry Tan: GBrain 成本对比帖 (2026/5/15) (<https://x.com/garrytan/status/2055310116325560647>): 用于参考维护者对 GBrain 成本效益的公开表述。
- Garry Tan: GBrain 重大更新帖 (2026/5/12) (<https://x.com/garrytan/status/2054055071017538028>): 用于确认 hot memory、facts extraction、72h 14 PRs 等迭代速度。
- Garry Tan: GBrain multiplayer/federated repos (2026/5/17) (<https://x.com/garrytan/status/2055884666264269177>): 用于确认 7 人团队多人使用、federated repos (个人+工作仓库并存)。
- Garry Tan: ZeroEntropy 嵌入默认 (2026/5/17) (<https://x.com/garrytan/status/2056119107133870149>): 参考来源。
- Garry Tan: GBrain 开发哲学 (2026/5/12) (<https://x.com/garrytan/status/2054036840521342996>): 参考来源。
- Garry Tan: SKILLIFY 循环 (2026/4/22) (<https://x.com/garrytan/status/2046981289031667961>): 参考来源。
- Garry Tan: GBrain 早期开源宣告 (2026/3/21) (<https://x.com/garrytan/status/2035366660451041387>): 参考来源。

本稿区分三类信息：

- 事实来源：公开仓库、公开解读文章和公开人物页；
- 分析推断：本书围绕跨境电商场景做出的系统化解释；
- 适用条件：多产品、多渠道、多数据源、有复盘节奏、有 Agent 化需求，并愿意维护结构化页面。

附录 E：后续学习路径

这本书是开源项目，会持续更新。后续的迭代、修正和新增内容会同步到 GitHub 仓库。

如果你已经跑通了前面的步骤，接下来会遇到更具体的问题：

- schema 怎么随业务扩展而不失控？
- 多人写入时怎么防止知识污染？
- 竞品分析 Skill 怎么写得更稳？
- 飞书 → Obsidian → GBrain 的自动化链路怎么搭？
- eval 怎么从 10 个问题扩展到业务级验收？
- 不同产品线、不同市场的 brain 要不要分开？

这些问题我们也在摸索。这本书给出的是到目前为止的判断框架，不是最终答案。

微信交流群：关注「Zach的进化笔记」公众号，回复”第二大脑”加入读者群。有问题直接聊，免费。

知识星球：如果你想看更多落地过程中的真实讨论、踩坑记录和其他卖家的实操案例，「Zach的进化笔记」知识星球是一群正在落地的人的持续讨论，不是课程。

如果你只想先做一个动作：从一页跨境业务复盘开始。不要等系统完美，先让下一次 Agent 能读到上一次人的判断。

附录 F：场景速查表

我想...	看哪章	关键操作	注意
理解为什么需要 AI 知识库	00 前言	读”你的 Agent 现在缺什么”	不需到
判断 RAG / LLM Wiki / GBrain 该选谁	02	对比表	看你具
理解三层基础概念	04	来源层 → 知识层 → 使用层	先看原
理清 Obsidian 和 GBrain 怎么配合	06	Obsidian 做编辑台，GBrain 做 Agent 记忆层	不是二
列出我的业务知识对象	08	写一页术语表	先覆盖
搭第一个 brain repo	15	按目录模板建文件夹	先写
写一条业务复盘	16, 附录 B	复制模板，填 Compiled Truth + Timeline	不要跟
做一次竞品分析	17	建竞品观察页	区分重
让 GBrain 跑起来	18	<code>gbrain import</code> → <code>embed --stale</code> → <code>query</code>	先 sy
检查 brain 是否好用	19	10 个 eval 问题	先人二
接入团队使用	20	Remote MCP + OAuth	先 re
理解双脑架构	21	双脑分工表	业务才
搭个人大脑	22	建目录、写画像页、接入模型	先写回
判断某个经验该不该入库	23	四条排除规则	实时数